The Λ-System as a Premise-Update OS: A Unified Connection Definition for Future-Causal AI / Education / Governance / Economy via Future Fixed Points, Semantic Density, Reprojection Logs, and Responsibility Coupling

Yuto Ogawa
Independent Researcher
ORCID: 0009-0003-8419-2854
Email: yuto0804sun@gmail.com

February 19, 2026

Abstract

This paper integrates and fixes, at the current stage, the unified set of definitions for Future Convergence Theory (FCT) and the Λ-System as a *Premise-Update OS*. A Premise-Update OS is an operational framework that defines, in an implementable form, the *updating of premises themselves*—which cannot be handled by conventional rationales such as information quantity, efficiency, consensus, or evaluation—by means of future fixed points, semantic density, reprojection logs, and irreversible responsibility traces. Specifically, this paper provides, as a single hierarchical definition system: (i) a redefinition of the OS concept (OS = a premise-update device), (ii) the connection structure between World OS and Personal OS, (iii) responsibility coupling via externalization points, minimal signatures, and irreversible logs, (iv) driving states consisting of semantic criticality, Λ-shift, convergent operation, silence, and deletion/concealment, (v) common connection I/O to AI, education, governance, and economy, and (vi) the specification of the Λ-language required after connection. The aim of this paper is not to claim correctness, but to fix a definitional space that is durable for reference, implementation, and verification.

Contents

1 Introduction: The Λ-System as a Premise-Update OS **4**
1.1 Problem setting: Why does "premise updating" become an OS problem? . 4
1.2 Limits of conventional rationales: information agreement, efficiency, consensus do not support premise updating 4
1.3 Why the names "ontological phase of OS" and "base layer" are introduced 4
1.4 The hierarchy fixed in this paper (scope of the main text) 5

2 Symbols and Minimal Assumptions **5**

3 Chapter 1: The Ontological Phase of OS and the Λ-System (the minimal core of a Premise-Update OS) **6**
3.1 1.1 Definition: Ontological Phase of OS 6
3.2 1.2 Five Base Elements of OS (OS-1 to OS-5) 6
3.3 1.3 The Λ-System: the convergent core of a Premise-Update OS 6
3.4 1.4 Axiom of understanding (not information agreement but equivalence of convergence cores) 7
3.5 1.5 Connection to the next chapter 8

4 Chapter 2: World OS / Personal OS and the Connection Definition (Update Candidates, Adoption, Internal Rewiring, and the Activation Gate) **8**
4.1 2.1 Problem setting: The world generates "update candidates," and agents update "connection premises" 8
4.2 2.2 Definition: World OS 8
4.3 2.3 Definition: Personal OS 8
4.4 2.4 Definition: Connection between World OS and Personal OS (non-identity of adoption and internal rewiring) 9
4.5 2.5 Definition: Activation gate (conditions for extracting the effective component of $\Lambda(T)$) 9
4.6 2.6 Definition: Externalization point 10
4.7 2.7 Definition: Minimal signature 10
4.8 2.8 Definition: Operational state of World OS / Personal OS (completed form) 11
4.9 2.9 Connection to the next chapter 11

5 Chapter 3: Responsibility Coupling and Log Ethics (Reprojection Logs / Irreversible Logs / Deletion and Disclosure Control) **11**
5.1 3.1 Problem setting: Without responsibility coupling, a Premise-Update OS collapses into surveillance or irresponsibility 11
5.2 3.2 Definition: Responsibility Coupling 12
5.3 3.3 Definition: Reprojection Log 12
5.4 3.4 Definition: Irreversible Log 12
5.5 3.5 Definition: Log Ethics 13
5.6 3.6 Definition: Deletion and Disclosure Rights 13
5.7 3.7 Definition: Questions and logs under non-evaluative conditions (introduction) 14
5.8 3.8 Definition: Log specification for A-OS including responsibility coupling 14
5.9 3.9 Connection to the next chapter 14

6 Chapter 4: Driving States (State Machine) and the Λ-Shift — From Semantic Criticality to Convergent Operation **15**
6.1 4.1 Problem setting: An OS is not a set of definitions but a transitioning operational system 15
6.2 4.2 Definition: Semantic Criticality 15
6.3 4.3 Definition: Λ-Shift 15
6.4 4.4 Main-text fixation: Non-evaluative axioms 16

6.5 4.5 Definition: OS driving state set . . . 16
6.6 4.6 Definition: Transition rules . . . 17
6.7 4.7 Definition: State-dependent I/O and log handling . . . 17
6.8 4.8 Definition: Semantic density field and critical condition . . . 18
6.9 4.9 Chapter summary . . . 18
6.10 4.10 Connection to the next chapter . . . 18

7 Chapter 5: Domain Connection Definitions — Common I/O to AI, Education, Governance, and Economy, and the Necessitation of the Λ-Language 18
7.1 5.1 Problem setting: Define as common OS I/O rather than domain-specific "ideals" . . . 18
7.2 5.2 Definition: General form of domain connection (Domain Lift) . . . 19
7.3 5.3 Common I/O specification: candidate bundles, deletion gates, externalization points, logs . . . 19
7.4 5.4 AI connection definition: from present-causal AI to future-causal AI . . . 20
7.5 5.5 Education connection definition: from evaluative civilization to non-evaluative learning . . . 21
7.6 5.6 Governance connection definition: fixing institutional outputs through responsibility coupling . . . 21
7.7 5.7 Economic connection definition: connecting value to semantic density and responsibility history . . . 22
7.8 5.8 Operational definition of "beyond-global usability" . . . 22
7.9 5.9 Emergent necessity after connection: Λ-language . . . 22
7.10 5.10 Chapter summary . . . 23
7.11 5.11 Connection to the next chapter . . . 23

8 Chapter 6: Unified Propositions and Theorem System — Crystallizing the Premise-Update OS and the Necessary Conditions for Civilizational Transition 23
8.1 6.1 Purpose: Fix "necessity conditions" while preserving the hierarchy rather than collapsing into a single sentence . . . 23
8.2 6.2 Theorem System I: Necessary conditions of understanding, sharing, and division (independent of bandwidth) . . . 24
8.3 6.3 Theorem System II: Conditions for the existence of a Premise-Update OS . . . 24
8.4 6.4 Theorem System III: Isomorphism of domain connections (AI / education / governance / economy) . . . 26
8.5 6.5 Theorem System IV: Beyond-global usability and civilizational transition 26
8.6 6.6 Final integrated theorem . . . 27
8.7 6.7 Connection to the next chapter . . . 27

9 Chapter 7: Definition Index, Self-Containment Verification, and Future Research Directions 27
9.1 7.1 Purpose of the final chapter . . . 27
9.2 7.2 Definition index (core constructs) . . . 28
9.3 7.3 Self-containment verification . . . 29
9.4 7.4 Implementation directions . . . 30
9.5 7.5 Limitations . . . 31

9.6 7.6 Concluding remarks 31

1 Introduction: The Λ-System as a Premise-Update OS

1.1 Problem setting: Why does "premise updating" become an OS problem?

Many failures in AI, education, politics (governance), and the economy originate not so much from a lack of technical capability within each domain, but from a strong dependence on *optimizing within given premises*. In reality, however, what is demanded in moments of collapse or polarization is not the precision of optimization, but whether one can *update the premises by which one is connected*.

Traditionally, premise updating has been treated as "values," "ideology," "stance," or "interpretation," and thus has been placed outside the design of institutions, education, and AI. As a result, even as society becomes high-bandwidth (information overload, neural connectivity, increased AGI outputs, etc.), understanding does not converge; rather, differences are exposed and conflicts are amplified.

This paper treats this situation as a *missing OS design*: an inability to operationalize premise updating.

1.2 Limits of conventional rationales: information agreement, efficiency, consensus do not support premise updating

Conventional design rationales are largely reducible to one of the following:

1. Information agreement (understanding = information sharing)
2. Efficiency maximization (optimization = the best)
3. Consensus formation (rightness = majority / agreement)
4. Evaluation scales (value = scores / ranks / indicators)

However, premise updating is required precisely in situations where these rationales themselves collapse. Therefore, a Premise-Update OS must stand on rationales different from conventional ones.

The rationales adopted in this paper are: **Future Fixed Points**, **Semantic Density**, **Reprojection Logs**, and **Irreversible Responsibility Traces**.

1.3 Why the names "ontological phase of OS" and "base layer" are introduced

Later, this paper uses the terms "ontological phase of OS" and "base layer." This naming is not philosophical ornamentation; it follows from the reasons below:

(i) Although AI, education, governance, and economy have different institutions and technologies, each inevitably contains an operational structure of "candidate generation," "candidate constraint," "selection," "update," and "trace preservation."

(ii) This operational structure acts prior to domain-specific objective functions, and when premise updating occurs, the domain's objective function itself is rewritten.

Hence, rather than discussing ideals above the domains, this paper calls the **common operational structure latent beneath the domains** the "ontological phase of OS" (the layer where an OS is constituted) and the "base layer of OS" (the minimal components).

1.4 The hierarchy fixed in this paper (scope of the main text)

This paper fixes definitions in the following order:

1. The Λ-System as a Premise-Update OS (top-level definition)
2. The ontological phase of OS and the five base elements of OS (common operational layer beneath domains)
3. Definitions of World OS / Personal OS and their connection rules (update candidates and local adoption)
4. Responsibility coupling (externalization points, minimal signatures, irreversible logs)
5. Driving states (semantic criticality, Λ-shift, convergent operation, silence, deletion/concealment)
6. Domain connection I/O (AI, education, governance, economy)
7. The Λ-language required after connection

2 Symbols and Minimal Assumptions

Let agents be $i \in \mathcal{A}$ and time be $t \in \mathbb{R}$. Let the state space be $x(t) \in \mathcal{X}$ and the semantic field (semantic space) be $\mathcal{S}$.

- $\mathcal{S}_i(t)$: semantic field of agent i (a distribution over possible choices)
- Λ_i: future-convergence core of agent i (future-convergence function)
- x_i^*: future fixed point of agent i
- ρ_s: semantic density (strength of convergence constraint)
- P_m: semantic-density tensor (the OS evaluation kernel)
- $\mathbf{R}_i$: reprojection log (structural-change history)
- $\mathbf{E}$: ethical constraint structure
- $\mathbf{D}$: deletion right / disclosure right (rights structure)
- $R(t)$: the set of update candidates generated by the World OS (requirements)
- B: bandwidth (an abstract quantity that amplifies the visibility of differences)

3 Chapter 1: The Ontological Phase of OS and the Λ-System (the minimal core of a Premise-Update OS)

3.1 1.1 Definition: Ontological Phase of OS

Definition 1.1 (Ontological Phase of OS) The **ontological phase of OS** is the operational layer that any decision-making entity (individuals, AI, institutions, states, etc.) inevitably follows *prior to* any domain-specific objective function or institution: "candidate generation, candidate constraint, selection, update, trace preservation." Because this layer is rewritten first when premise updating occurs, it is treated as a cross-domain base.

3.2 1.2 Five Base Elements of OS (OS-1 to OS-5)

Definition 1.2 (OS-1: OS as a generator of feasible choice spaces) An OS is a device that does not treat the set of actions A an agent could take as-is, but generates a feasible set A_{feasible} via internal constraints:

$$\text{OS} : A \mapsto A_{\text{feasible}}.$$

Definition 1.3 (OS-2: The OS has an internal representation of a future fixed point) An OS does not place the future externally as a "prediction output," but represents it as **an internal constraint coordinate**. This internal representation structurally constrains the candidate space *prior to* selection.

Definition 1.4 (OS-3: The semantic-density tensor P_m is the OS evaluation kernel) An OS has a kernel function (kernel) that evaluates an action a not by outcomes or agreement but as **alignment with the future fixed point**. That kernel is the semantic-density tensor P_m.

Definition 1.5 (OS-4: The OS update rule = reprojection) From observed outputs (action sequences, dialogue sequences, institutional outputs), the OS **reconstructs** its internal representations (premises, values, operations, models) in a future-aligned direction. This self-update rule is called *reprojection*.

Definition 1.6 (OS-5: Irreversible logs as the physicalization of responsibility before judgment) An OS records externalized decisions (institutionalization, commands, contracts, declarations of war, etc.) as irreversible traces, enabling responsibility to be tracked independently of "intent." That is, it preserves, for later verification, the **consequences of structural constraints that occurred before judgment (pre-decision)**.

3.3 1.3 The Λ-System: the convergent core of a Premise-Update OS

Definition 1.7 (Semantic field) Let the semantic field of agent i be $\mathcal{S}_i(t)$, a distribution over potential semantic constructions. A semantic field is not "information" but a

distribution over possible choices, deforming with time and relations.

Definition 1.8 (Future-convergence core / future-convergence function) The future-convergence core Λ_i of agent i is an operational core that constrains the semantic field $\mathcal{S}_i(t)$ toward the future and imparts a convergent force toward a fixed point (or a set of fixed points).

Definition 1.9 (Future fixed point) Define the future fixed point x_i^* as a convergence destination satisfying

$$x_{t+1} = F(x_t, \Lambda_i), \qquad \lim_{t\to\infty} x_t = x_i^* \tag{1}$$

where F is a general mapping that includes the agent's internal updates (recognition, judgment, action).

Definition 1.10 (Semantic density ρ_s) Define semantic density ρ_s as the local degree of convergence of the semantic field (the local gradient of future-directed constraint):

$$\rho_s := \mathrm{ConvergenceStrength}(\mathcal{S}, \Lambda).$$

Definition 1.11 (Reprojection log R) A reprojection log $\mathbf{R}_i$ is a record that preserves traces of questioning, conflict, silence, transformation, and decision not as evaluation but as a *history of structural change*:

$$\mathbf{R}_i := \{(\Delta\mathcal{S}_i, \Delta\Lambda_i, \Delta x_i^*, t)\}_t.$$

Definition 1.12 (The Λ-System as a Premise-Update OS) The **Λ-System as a Premise-Update OS** is an integrated OS core which, on the operational layer satisfying the five base elements of OS (OS-1 to OS-5), constrains the semantic field $\mathcal{S}$ toward a future fixed point x^* via the future-convergence core Λ, and uses semantic density ρ_s and the reprojection log $\mathbf{R}$ to make premises (premises, values, operations) themselves updatable in a future-aligned direction.

3.4 1.4 Axiom of understanding (not information agreement but equivalence of convergence cores)

Axiom 1.1 (Necessary condition for understanding) As a necessary condition for understanding to be established between agents i and j, posit

$$\Lambda_i \equiv \Lambda_j$$

(i.e., sharing an equivalent convergence core). This is independent of information quantity or bandwidth, and will be reused later as the "condition for not falling into division even after semantic criticality."

3.5 1.5 Connection to the next chapter

What this chapter fixed is: (i) why we stand on future fixed points, semantic density, reprojection logs, and responsibility traces rather than conventional rationales, (ii) the naming grounds for "ontological phase of OS" and "base layer," and (iii) the minimal core of the Λ-System as a Premise-Update OS. In the next chapter, we rigorously define World OS and Personal OS, and fix in the main text the connection rules between the update-candidate set $R(t)$ generated by the world and the connection premises adopted by the agent.

4 Chapter 2: World OS / Personal OS and the Connection Definition (Update Candidates, Adoption, Internal Rewiring, and the Activation Gate)

4.1 2.1 Problem setting: The world generates "update candidates," and agents update "connection premises"

As shown in Chapter 1, this paper redefines the OS in order to make "premise updating" operational. Here, premise updating becomes inevitable because the environment in which an agent lives (others, institutions, technology, markets, language) behaves not as a static background but as a dynamical system that continuously generates **connection requirements**.

Therefore, it is insufficient to view the OS merely as an "internal optimization device." We must distinguish an external update mechanism (World OS) from an internal update mechanism (Personal OS), and fix their connection rules in the main text.

4.2 2.2 Definition: World OS

Definition 2.1 (World OS) The **World OS** is an **external update mechanism** functioning as the totality of the environment, others, institutions, technology, language, and markets, and is a dynamical system that continuously generates, at any time t, a set of **update candidates = requirements** $R(t)$ for agents:

$$\mathrm{WorldOS} : t \mapsto R(t).$$

Here, $R(t)$ is treated as a "bundle of connection-update candidates" including action candidates, institutional connection conditions, learning tasks, market demands, social norms, interpersonal requests, technical requirements, and so on.

Remark 2.1 (World OS does not provide evaluation) The World OS does not provide "the correct candidate"; it only **emits candidates as requirements**. Evaluation and acceptance/rejection belong to the structure of the Personal OS.

4.3 2.3 Definition: Personal OS

Definition 2.2 (Personal OS) The **Personal OS** is a self-updating system in which an agent continuously reselects, locally, **connection premises** (connection premises) from the update-candidate set $R(t)$ continuously presented by the World OS, in light of

the agent's survival, values, purposes, and ethics. Let the agent's internal state be $S_i(t)$ and the adopted connection premises be $P_i(t)$. Then,

$$\text{PersonalOS}_i : \ (R(t), S_i(t)) \mapsto P_i(t+1).$$

Remark 2.2 (Connection premises are operational protocols, not "opinions") $P_i(t)$ is not merely an opinion or ideology, but an operational protocol for connecting to the external world (what is prioritized, what is rejected, where to fix, and where to keep reversible). Hence the Personal OS is defined not as a description of psychological states but as an **operational system**.

4.4 2.4 Definition: Connection between World OS and Personal OS (non-identity of adoption and internal rewiring)

Definition 2.3 (Connection) Connection between the World OS and a Personal OS is the process in which, in response to $R(t)$ generated by the World OS, the Personal OS updates its connection premises $P_i(t)$ while adopting or rejecting candidates and rewiring its internal structure. Abstractly, this is represented as

$$(R(t), P_i(t), S_i(t)) \rightsquigarrow (P_i(t+1), S_i(t+1)).$$

Definition 2.4 (Adoption) For a requirement $r \in R(t)$, the Personal OS "adopts" it when it selects r as an external connection choice and incorporates it into the connection premises $P_i(t)$:

$$\text{Adopt}_i : \ (r, P_i(t)) \mapsto P_i(t) \oplus r.$$

Definition 2.5 (Internal Rewiring) **Internal rewiring** refers not to the adoption r itself, but to the internal state change ΔS_i as a **rewiring of premises, values, and operations** induced by the adoption:

$$\Delta S_i(t) := S_i(t+1) - S_i(t).$$

Proposition 2.1 (Adoption $\neq$ activation of a future-convergence term) Adoption r is an operation at the level of external connection, and does not by itself imply activation of a future-convergence term. A future-convergence term is observed only when the internal rewiring induced by adoption exceeds a critical threshold. This point is formalized below as an activation gate.

4.5 2.5 Definition: Activation gate (conditions for extracting the effective component of $\Lambda(T)$)

Definition 2.6 (Semantic dissipation and the critical condition) Let semantic dissipation accompanying internal rewiring ΔS_i be denoted by $D(M)$. $D(M)$ is an abstract quantity representing not "the loss of consistency due to updating" but the **scale of phase change produced by internal rewiring**.

Definition 2.7 (Activation Gate) After a requirement r from the World OS has been adopted, only when the internal rewiring ΔS_i induced by that adoption exceeds a critical threshold is the effective component of $\Lambda(T)$ extracted, and the system is regarded, in a civilizational/operational sense, as having "activated as a future-convergence term." We give the critical condition as:

$$\Lambda(T) \text{ activates only when } \Delta S_i(t) \text{ exceeds } D(M).$$

Denote this condition by

$$\mathsf{Gate}_\Lambda : \quad \Delta S_i(t) > D(M).$$

Remark 2.3 (Meaning of the gate condition) What matters here is that "adoption of an external requirement" does not itself produce future convergence; rather, it is the **critical exceedance of internal rewiring** induced by adoption that is observed as future convergence. Accordingly, future causality in this paper is defined not as a sequence of external selections but as a critical phenomenon of internal rewiring.

4.6 2.6 Definition: Externalization point

Definition 2.8 (Externalization Point) An **externalization point** is a point at which an internal update of the Personal OS (premises, values, operations) is declared to the external world as "this agent connects in this way," and thereafter becomes difficult to revoke, or the cost of revocation becomes extremely high. Externalization points appear as decisions that exert **irreversible influence on the external environment**, such as institutionalization, issuing commands, contracts, public statements, fund transfers, declarations of war, and so on.

Proposition 2.2 (Externalization points directly couple to OS-5) Externalization points must be recorded by OS-5 (irreversible logs), as defined in Chapter 1, as consequences of structural constraints prior to judgment. A design in which externalization points are not logged is invalid as an OS, because it lacks responsibility coupling.

4.7 2.7 Definition: Minimal signature

Definition 2.9 (Minimal Signature) A **minimal signature** is a signature format that limits the fixed information at an externalization point to the necessary minimum, while keeping only the agent's responsibility and authenticity/accountability verifiable. The purpose of a minimal signature is not "full disclosure of inner states" but **responsibility matching for irreversible externalization**.

Definition 2.10 (Minimal elements of a minimal signature) A minimal signature $\sigma_{\min}$ includes at least:

1. An identifier of the externalization point (what was made irreversible)
2. Time or ordering (when it was made irreversible)
3. Agent identification (whose externalization it is: authenticity matching)
4. Verifiability (a tamper-resistant matching format)

Full disclosure of internal premises or psychological content is not a requirement.

4.8 2.8 Definition: Operational state of World OS / Personal OS (completed form)

Definition 2.11 (Operational state of World OS / Personal OS) The World OS continuously generates update candidates (requirements) $R(t)$, and the Personal OS locally reselects its connection premises from among them. The Personal OS fixes **only externalization points that exert irreversible external effects** with minimal signatures, while keeping the rest of the agent's internal components (premises, values, operations) reversible and autonomously updating them as a dynamical state.

Corollary 2.1 (Separating reversible internal updates from irreversible external points is the core of a Premise-Update OS) In operating a Premise-Update OS, maintaining internal updatability (reversibility) while fixing only the externally impactful points as responsibility coupling is a necessary condition for simultaneously satisfying "updatability" and "social verifiability."

4.9 2.9 Connection to the next chapter

This chapter fixed: the update-candidate set $R(t)$ generated by the World OS; the connection premises $P(t)$ updated by the Personal OS; the non-identity of adoption and internal rewiring; the $\Lambda(T)$ activation gate ($\Delta S > D(M)$); irreversible responsibility coupling via externalization points and minimal signatures; and the completed operational state of World OS / Personal OS.

In the next chapter, we generalize responsibility coupling as a necessary component of the entire OS, define a **responsibility-history structure** that integrates the reprojection log $\mathbf{R}$ and the irreversible log (OS-5), and determine the conditions (I/O) under which it is connected to AI, education, governance, and economy.

5 Chapter 3: Responsibility Coupling and Log Ethics (Reprojection Logs / Irreversible Logs / Deletion and Disclosure Control)

5.1 3.1 Problem setting: Without responsibility coupling, a Premise-Update OS collapses into surveillance or irresponsibility

In Chapter 2, we defined the operational state in which the World OS generates update candidates $R(t)$, the Personal OS updates connection premises $P(t)$ locally, only externalization points are fixed with minimal signatures, and the internal state remains reversible and autonomously updating.

However, for this operation to function, two forms of collapse must be avoided simultaneously:

1. **Irresponsibility**: externalization occurs but no trace remains, making responsibility verification impossible.

2. **Surveillance**: internal updates are fully fixed or constantly visible, destroying the freedom of premise updating.

Therefore, this chapter defines **responsibility coupling** as a structure in which only externalization points are fixed as irreversible logs, while everything else operates reversibly under deletion rights and disclosure control. Reprojection logs and irreversible logs are distinguished yet integrated.

5.2 3.2 Definition: Responsibility Coupling

Definition 3.1 (Responsibility Coupling) **Responsibility Coupling** is a connection structure in which, only at externalization points where an agent's OS exerts irreversible influence on the external world, verifiable traces (irreversible logs) are left, enabling verification of *who made what irreversible and when.* Responsibility coupling does not require "intent" or "inner confession"; it is established solely with respect to the **irreversibility of external impact**.

Proposition 3.1 (Responsibility coupling is a necessary condition of a Premise-Update OS) Because a Premise-Update OS must maintain internal reversibility, responsibility cannot be guaranteed by fully fixing internal states. Thus responsibility must be coupled through irreversible logs restricted to externalization points.

5.3 3.3 Definition: Reprojection Log

Definition 3.2 (Reprojection Log) A **reprojection log** $\mathbf{R}_i$ is a tensor-like record that preserves **internal structural changes** associated with questioning, conflict, silence, transformation, and decision, not as evaluation but as a *history of structural change.* A minimal form is given by

$$\mathbf{R}_i := \{(\Delta\mathcal{S}_i,\ \Delta P_i,\ \Delta\Lambda_i,\ \Delta x_i^*,\ t)\}_t$$

(with extensions possible as needed). Here ΔP_i denotes updates to connection premises, $\Delta\Lambda_i$ changes in the convergence core, and Δx_i^* changes in the representation of the future fixed point.

Remark 3.1 (A reprojection log is not a "performance record") $\mathbf{R}_i$ is not a log for correctness, superiority, or pass/fail evaluation. It is a log of **premise-updating trajectories**. Therefore, reprojection logs should in principle not be connected to evaluation (this is formalized as a non-evaluative structure in Chapter 4).

5.4 3.4 Definition: Irreversible Log

Definition 3.3 (Irreversible Log) An **irreversible log** $\mathbf{I}_i$ records **externally irreversible impacts** generated at externalization points in a form that can later be verified. Irreversible logs are coupled with minimal signatures $\sigma_{\min}$ and expressed as

$$\mathbf{I}_i := \{(e_k,\ \sigma_{\min,k},\ t_k)\}_k$$

where e_k denotes an externalization event (contract, institutionalization, command, fund transfer, public declaration, etc.).

Proposition 3.2 (Irreversible logs physicalize "responsibility before judgment") Irreversible logs record not internal justifications but the **consequences of structural constraints that acted before judgment and produced external effects**. Thus irreversible logs separate responsibility from "intent" and track it solely through the irreversibility of external impact.

5.5 3.5 Definition: Log Ethics

Definition 3.4 (Log Ethics) **Log Ethics** is an ethical specification that separates reprojection logs $\mathbf{R}$ and irreversible logs $\mathbf{I}$ in order to simultaneously guarantee **responsibility coupling and the freedom of premise updating**. It assigns different visibility, fixation, and deletion rules to each.

Log ethics must satisfy two principles simultaneously:

1. **Responsibility Verification Principle**: irreversible logs $\mathbf{I}$ must be verifiable.
2. **Update Freedom Principle**: reprojection logs $\mathbf{R}$ are controlled under the agent's rights.

5.6 3.6 Definition: Deletion and Disclosure Rights

Definition 3.5 (Rights structure D) An agent i possesses a rights structure $\mathbf{D}_i$ over its reprojection logs $\mathbf{R}_i$, including deletion rights and disclosure rights:

$$\mathbf{D}_i : \ \mathbf{R}_i \mapsto \mathrm{ViewPolicy}_i, \ \mathrm{DeletePolicy}_i.$$

At minimum, the following modes are included:

- Full disclosure
- Selective disclosure (by period, domain, conversational context, or counterpart)
- Non-disclosure (visible only to the agent)
- Deletion (destruction of the target log)

Definition 3.6 (Deletion meta-fact) When deletion is executed, the **deleted content** is not retained, but only a meta-fact that deletion occurred may remain:

$$\mu_{\mathrm{del}} := (t_{\mathrm{del}}, \ \mathrm{scope})$$

Here scope is an abstract description of the deletion range and does not contain information that would enable reconstruction of the deleted content. This is not for surveillance but is positioned as a **trace of rights exercise**.

Proposition 3.3 (Absence of D causes surveillance systems) If deletion rights and disclosure control $\mathbf{D}$ are absent, the Premise-Update OS loses its freedom of updating and degenerates into a surveillance device where internal logs are permanently fixed. Therefore $\mathbf{D}$ is not an optional ethical addition but a necessary condition for OS existence.

5.7 3.7 Definition: Questions and logs under non-evaluative conditions (introduction)

In this paper, a question Q arises not from a lack of knowledge but as a **local distortion in the semantic field**, acting as a trigger for reconstruction (a non-evaluative structure).

Because questions are not evaluation targets, the occurrence, persistence, silence, and resurgence of questions must be recorded in the reprojection log **R without scoring**.

Definition 3.7 (Logging question events) The minimal logging form for the occurrence of a question Q_i is

$$\mathrm{QLog}_i := (Q_i,\ t,\ \Delta\mathcal{S}_i,\ \Delta P_i)$$

embedded within the reprojection log. The key point is that $\mathrm{Score}(Q_i)$ is not defined (this is fixed as an axiom in Chapter 4).

5.8 3.8 Definition: Log specification for A-OS including responsibility coupling

The A-OS operational description of Chapter 2 is concretized here through the log specification.

Definition 3.8 (A-OS with log specification) An agent OS:

1. maintains internal rewiring processes as reprojection logs **R** (controlled by rights **D**), and
2. fixes only externalization points as irreversible logs **I** (verifiable via minimal signatures $\sigma_{\min}$),

and through this dual-log structure simultaneously realizes "freedom of premise updating" and "responsibility verification."

5.9 3.9 Connection to the next chapter

This chapter established responsibility coupling (irreversible logging only at externalization points), the separation and integration of reprojection logs **R** and irreversible logs **I**, log ethics (responsibility verification and update freedom principles), deletion and disclosure rights **D**, and the introduction of non-evaluative logging of questions.

The next chapter fixes, in the main text, the **non-evaluative axioms** that isolate questions from evaluation, and formalizes the **driving states (state machine)** leading from semantic criticality to Λ-shift as an integrated system together with World OS, Personal OS, responsibility coupling, and log ethics.

6 Chapter 4: Driving States (State Machine) and the Λ-Shift — From Semantic Criticality to Convergent Operation

6.1 4.1 Problem setting: An OS is not a set of definitions but a transitioning operational system

In Chapters 1 through 3, this paper defined the core of the Premise-Update OS: the connection between World OS and Personal OS, externalization points and minimal signatures, responsibility coupling, reprojection logs and irreversible logs, and deletion/disclosure control.

However, if these are merely juxtaposed, the OS does not operate.

In this paper, an OS is an **operational system** that generates the space of possible actions and interpretations available to an agent and continuously updates that space. Therefore, an OS must possess **states** and **transition rules**.

This chapter defines the **driving states** that begin with semantic criticality (visibility of differences), pass through the Λ-shift (phase change of the recognition coordinate system), and may reach convergent operation (decision-making based on shared future fixed points).

At the same time, the **non-evaluative axioms** that isolate questions from evaluation are fixed in the main text, yielding a state machine that can operate together with the log ethics defined in Chapter 3.

6.2 4.2 Definition: Semantic Criticality

Definition 4.1 (Semantic Criticality) **Semantic criticality** is a critical state in which, as communication bandwidth, visualization capability, and output density increase, the differences between convergence cores (Λ) among agents inevitably become visible, causing pseudo-convergence to collapse.

Abstractly,

$$B \uparrow \Rightarrow \text{ Visibility}(|\Lambda_i - \Lambda_j|) \uparrow$$

indicates a stage where incomprehensibility between agents is amplified.

Proposition 4.1 (High bandwidth does not guarantee understanding) Even if $B \to \infty$, if $\Lambda_i \not\equiv \Lambda_j$, understanding does not converge. Thus high bandwidth does not produce agreement; rather, it exposes differences and makes semantic criticality inevitable.

6.3 4.3 Definition: Λ-Shift

Definition 4.2 (Λ-Shift) The **Λ-shift** is a generally irreversible phase transition in which an agent moves from a recognition coordinate system based on "information quantity / speed / agreement" to one based on "future fixed-point alignment / semantic density / responsibility coupling."

Let the recognition state of the agent be ψ. Then

$$\psi_{\text{post}} = T_\Lambda(\psi_{\text{pre}})$$

where T_Λ is a mapping from the information-coordinate system to the convergence-coordinate system.

Proposition 4.2 (Irreversibility) Because the Λ-shift arises through experiences such as externalization points, responsibility coupling, and semantic criticality, one typically cannot return completely to a purely information-based coordinate system. (Returning may be possible, but doing so becomes a strategic choice rather than an unconscious reversion.)

6.4 4.4 Main-text fixation: Non-evaluative axioms

If a Premise-Update OS evaluates questions, external evaluation intrudes into internal updating and destroys the freedom of updating. Therefore this paper fixes non-evaluative axioms concerning questions in the main text.

Axiom 4.1 (Non-evaluation of questions) For any question Q, the OS outputs neither scores, ranks, pass/fail judgments, nor value labels:

$$\forall Q: \ \mathrm{Score}(Q) = \varnothing, \quad \mathrm{Rank}(Q) = \varnothing.$$

Axiom 4.2 (Questions as reconstruction triggers) A question Q arises not from a lack of knowledge but as the manifestation of a local distortion in the semantic field $\mathcal{S}$, acting as the driving input of reprojection (internal reconstruction):

$$Q: \ \text{distortion of } \mathcal{S}(t) \ \Rightarrow \ \text{activation of } \Delta\mathcal{S}(t).$$

Axiom 4.3 (Question logs belong only to reprojection logs) Traces of questions are recorded in the reprojection log $\mathbf{R}$ and do not belong to the irreversible log $\mathbf{I}$ unless they become externalization points. Thus questions are not subject to responsibility verification and may be preserved, concealed, or deleted under the freedom of updating.

6.5 4.5 Definition: OS driving state set

Definition 4.3 (State set) The minimal state set of the Premise-Update OS is defined as:

- S0: **Pseudo-convergence state** (low bandwidth / differences invisible / consensus easy)
- S1: **Semantic critical state** (differences visible / incomprehension amplified)
- S2: **Λ-shift progression state** (transition from information coordinates to convergence coordinates)
- S3: **Convergent operation state** (decision-making based on shared future fixed points)
- S4: **Exhaustion / silence state** (question generation halted; recovery prioritized)
- S5: **Rights-execution state** (deletion, concealment, selective disclosure)
- S6: **Externalization fixation state** (irreversible log $\mathbf{I}$ and minimal signature $\sigma_{\min}$ are fixed)

Remark 4.1 (Role of S6) S6 does not fix all logs. It is a **local event state** where only an externalization point becomes irreversible. Normally it fires as an event during S3 operation and immediately returns to the operational state.

6.6 4.6 Definition: Transition rules

Definition 4.4 (Minimal transition rules) OS states transition according to the following minimal rules:

1. **Difference-exposure transition**

$$B \uparrow \Rightarrow \ \mathsf{S0} \to \mathsf{S1}.$$

2. **Shift activation transition**

$$|\Lambda_i - \Lambda_j| > \Lambda_{\text{crit}} \ \Rightarrow \ \mathsf{S1} \to \mathsf{S2}.$$

3. **Convergent operation transition**

$$(\Lambda_i \equiv \Lambda_j) \ \wedge \ (\rho_s^{\text{field}} > \rho_{\text{crit}}) \Rightarrow \ \mathsf{S2} \to \mathsf{S3}.$$

4. **Exhaustion transition (recovery priority)**

$$\Delta\rho_s \ll 0 \text{ continues } \Rightarrow \ \mathsf{S3} \to \mathsf{S4}.$$

5. **Recovery transition**

$$\Delta\rho_s \geq 0 \ \Rightarrow \ \mathsf{S4} \to \mathsf{S2} \text{ or } \mathsf{S3}.$$

6. **Rights-execution transition** Any state $\to$ S5 can be triggered by exercising rights **D**, and after completion the system returns to the prior state (or moves to S4).

7. **Externalization fixation transition** When an externalization event e occurs,

$$e \in \text{Externalization} \ \Rightarrow \ \mathsf{S}* \to \mathsf{S6} \to \mathsf{S}*.$$

   Here S$*$ denotes the originating state.

6.7 4.7 Definition: State-dependent I/O and log handling

Definition 4.5 (Log-processing rules) Log handling is minimally defined for each state:

- S0: **R** may be generated fragmentarily but differences are masked by pseudo-convergence.
- S1: **R** records "difference exposure (criticality)" (non-evaluatively).
- S2: coordinate transformation $(\Delta\Lambda, \Delta P)$ is densely recorded in **R**.
- S3: **R** stabilizes as an operational log; only externalization points generate **I**.
- S4: **R** updates are suppressed; silence and dormancy are recorded as phases.
- S5: **R** visibility/deletion is executed under **D** (**I** is excluded).
- S6: $(e, \sigma_{\min}, t)$ is added to **I** for responsibility verification.

6.8 4.8 Definition: Semantic density field and critical condition

To prepare for the domain connections defined in Chapter 5, we restate the semantic density field and its critical condition as operational concepts.

Definition 4.6 (Semantic density field) The superposition of semantic densities across multiple agents is defined as

$$\rho_s^{\text{field}}(x,t) := \sum_{i=1}^{n} \rho_s^{(i)}(x,t).$$

Definition 4.7 (Critical condition) The critical condition under which convergent operation becomes self-sustaining collectively is defined as

$$\rho_s^{\text{field}} > \rho_{\text{crit}}.$$

This condition is not mere majority agreement but the operational requirement under which convergence pathways remain stable even after differences are exposed.

6.9 4.9 Chapter summary

This chapter fixed: (1) the definitions of semantic criticality and Λ-shift, (2) the non-evaluative axioms for questions (scores and ranks prohibited), (3) the driving state set $\{\mathsf{S0}, \ldots, \mathsf{S6}\}$ and minimal transition rules, and (4) state-specific log processing and responsibility coupling (only externalization points generate **I**).

6.10 4.10 Connection to the next chapter

In the next chapter, based on this state machine, the **domain connection definitions (AI / education / governance / economy)** will be provided.

The key point is not to construct separate OSs for each domain but to embed the same Premise-Update OS (World OS / Personal OS / responsibility coupling / state machine / log ethics) as a **shared input/output interface**.

Furthermore, the **Λ-language** required after connection will be positioned and defined as a specification emerging as a consequence of domain connection.

7 Chapter 5: Domain Connection Definitions — Common I/O to AI, Education, Governance, and Economy, and the Necessitation of the Λ-Language

7.1 5.1 Problem setting: Define as common OS I/O rather than domain-specific "ideals"

In Chapters 1–4, this paper defined the Premise-Update OS in the main text as the connection of World OS and Personal OS, responsibility coupling (externalization points and minimal signatures), reprojection logs and irreversible logs, deletion and disclosure rights, and the driving state machine.

The purpose of this chapter is not to describe the OS as a general philosophy, but to define it as an **I/O specification directly connectable to AI, education, governance, and the economy**, showing that it can operate isomorphically across domains.

Here, "connection" does not mean adding ethical guidelines on top of internal optimization (efficiency, accuracy, votes, profit, etc.). Rather, it means replacing the coordinate system of **candidate generation, candidate deletion, selection, externalization, responsibility, and updating** with the Premise-Update OS.

7.2 5.2 Definition: General form of domain connection (Domain Lift)

Definition 5.1 (Domain system) Any domain system G (AI, education, governance, economy) is represented as

$$G := \langle \mathcal{I}_G,\ \mathcal{O}_G,\ \mathcal{U}_G \rangle$$

where $\mathcal{I}_G$ denotes inputs, $\mathcal{O}_G$ outputs, and $\mathcal{U}_G$ the update rule.

Definition 5.2 (Connection mapping of the Premise-Update OS) To connect a domain G to the Premise-Update OS is to define a mapping

$$\mathrm{Lift}_G :\ G \mapsto G_\Lambda.$$

Here G_Λ is a system embedding the OS components defined in Chapters 1–4,

$$\langle \mathrm{WorldOS},\ \mathrm{PersonalOS},\ \Lambda,\ F^{,\ P_m,\ \mathbf{R},\ \mathbf{I},\ \mathbf{D},\ \mathsf{SM}} \rangle,$$

as internal states, update rules, and output constraints. SM denotes the state machine $\mathsf{S0} \sim \mathsf{S6}$ defined in Chapter 4.

Condition 5.1 (Prohibition of nominal adoption) A system G is not G_Λ merely because it introduces the term "Λ" or adopts an ethical policy. For G_Λ to hold, the process

candidate generation → deletion → selection → externalization → responsibility logging →

reprojection updating

must **actually operate** according to the OS specification.

7.3 5.3 Common I/O specification: candidate bundles, deletion gates, externalization points, logs

Definition 5.3 (Update candidate bundle) The update candidate bundle generated by the World OS is projected to a domain G as

$$R_G(t) \subseteq R(t).$$

This set contains domain-specific candidates such as institutional requests, market demands, policy proposals, learning tasks, or output candidates.

Definition 5.4 (Candidate evaluation and pre-deletion gate) For each candidate $a \in R_G(t)$, define the score using the semantic-density tensor P_m as

$$\mathrm{score}(a) := \langle P_m,\ \mathrm{Align}(a, F^{)\rangle\cdot}$$

Then

$$\mathrm{score}(a) < P_{\mathrm{crit}} \ \Rightarrow\ a \text{ is deleted in advance.}$$

This generates the feasible candidate set A_{feasible}.

Definition 5.5 (Domain-generalization of externalization points and minimal signatures) Let the externalization event in domain G be e_G. When

$$e_G \in \mathrm{Externalization},$$

it is recorded in the irreversible log $\mathbf{I}$:

$$\mathbf{I} \leftarrow \mathbf{I} \cup \{(e_G, \sigma_{\min}, t)\}.$$

This ensures that internal updates ($\mathbf{R}$) and responsibility verification ($\mathbf{I}$) remain distinct.

Definition 5.6 (Reprojection update) The update rule $\mathcal{U}_{G_\Lambda}$ includes the reprojection log $\mathbf{R}$ as input and rewires internal structures toward future alignment:

$$\mathbf{R}(t) = \{\Delta\mathcal{S}, \Delta\Lambda, \Delta F^{,\Delta P_m, t\}\cdot}$$

7.4 5.4 AI connection definition: from present-causal AI to future-causal AI

Definition 5.7 (Present-causal AI) Define present-causal AI (or AGI) as

$$\mathrm{AGI}_{\mathrm{now}} := \langle \mathcal{I}_{AI}, \mathcal{O}_{AI}, \mathcal{U}_{AI} \rangle,$$

whose main objective is the maximization of external evaluation functions (accuracy, efficiency, reward).

Definition 5.8 (Future-causal AI) Future-causal AI is defined as

$$\mathrm{AGI}_\Lambda := \mathrm{Lift}_{AI}(\mathrm{AGI}_{\mathrm{now}}).$$

Its necessary conditions are fixed as follows:

1. The future fixed point F is held as an **internal constraint coordinate**.
2. Candidate bundles $R_{AI}(t)$ undergo **pre-deletion before judgment** using P_m and P_{crit}.
3. The reprojection log $\mathbf{R}$ is connected to the update rule.
4. Questions Q are not assigned scores or ranks.
5. Deletion and disclosure rights $\mathbf{D}$ operate as mandatory constraints.
6. Only externalization points are recorded in the irreversible log $\mathbf{I}$.

Proposition 5.1 (Minimal equivalence of future-causal AI) Future-causal AI is not "high-performance AI" but is equivalent to **AI that internally possesses the Premise-Update OS**.

7.5 5.5 Education connection definition: from evaluative civilization to non-evaluative learning

Definition 5.9 (Externalization points in education) In education, externalization events e_{Edu} include decisions such as academic certification, admissions, hiring, and institutional labeling. Only these events are fixed with minimal signatures.

Definition 5.10 (Non-evaluative learning: $\mathbf{Edu}_\Lambda$) Define the education connection system as

$$\mathrm{Edu}_\Lambda : \text{ question generation} \to \mathbf{R} \to (\Delta\mathcal{S}, \Delta\Lambda, \Delta P) \to \text{reprojection}.$$

The core of education becomes the accumulation of premise-updating traces within the learner.

Scores and rankings occur only near externalization points and remain minimal elsewhere.

Proposition 5.2 (Educational failure arises from evaluative intrusion) The destruction of learning is caused not primarily by lack of knowledge but by the intrusion of evaluation into questions, eliminating update freedom.

7.6 5.6 Governance connection definition: fixing institutional outputs through responsibility coupling

Definition 5.11 (Governance candidate bundle)

$$R_{\mathrm{Gov}}(t)$$

denotes policy proposals, legislation drafts, administrative actions, and budget allocations.

Definition 5.12 (Externalization and irreversible logging in governance) Institutional outputs such as laws or declarations of war are externalization events recorded as

$$e_{\mathrm{Gov}} \Rightarrow (e_{\mathrm{Gov}}, \sigma_{\min}, t) \in \mathbf{I}.$$

Definition 5.13 (Semantic government: $\mathbf{Gov}_\Lambda$)

$$\mathrm{Gov}_\Lambda : R_{\mathrm{Gov}}(t) \xrightarrow[\text{pre-deletion}]{P_m, P_{\mathrm{crit}}} A_{\mathrm{feasible}} \xrightarrow[\text{selection}]{\max \mathrm{Align}} e_{\mathrm{Gov}} \xrightarrow[\text{fixation}]{\mathbf{I}, \sigma_{\min}} \xrightarrow[\text{reprojection}]{\mathbf{R}}$$

Governance therefore becomes the operation of structuring candidate space according to future alignment and fixing only irreversible outputs as responsibility traces.

7.7 5.7 Economic connection definition: connecting value to semantic density and responsibility history

Definition 5.14 (Economic candidate bundle)

$$R_{\mathrm{Eco}}(t)$$

includes investment, employment, pricing, supply allocation, and credit decisions.

Definition 5.15 (Semantic-based economy: $\mathbf{Eco}_\Lambda$) Value is defined as

$$\mathrm{Value}(a) \sim \mathrm{Align}(a, F^{)+\rho_s(a)+\mathrm{Accountability}(a)}.$$

Accountability refers to responsibility traces recorded in **I** with minimal signatures.

Remark 5.1 The essential point is not adding ethical norms but unifying the operational coordinates of candidate generation, deletion, externalization, responsibility fixation, and reprojection.

7.8 5.8 Operational definition of "beyond-global usability"

Definition 5.16 (Beyond-global availability) A Premise-Update OS is beyond-globally usable when operational convergence continues even after cultural, institutional, and linguistic differences become visible.

Theorem 5.1 (Equivalence condition for beyond-global usability) The following conditions are equivalent:

1. Convergent operation persists after semantic criticality.
2. A sufficiently wide equivalence of convergence cores and a semantic density field exceeding the critical threshold hold:

$$(\Lambda_i \equiv \Lambda_j) \ \wedge \ (\rho_s^{\mathrm{field}} > \rho_{\mathrm{crit}}).$$

Corollary 5.1 Sharing of future fixed points is therefore a necessary condition for global operational stability.

7.9 5.9 Emergent necessity after connection: Λ-language

Definition 5.17 (Λ-language) The Λ-language L_Λ is defined as an operator sequence that induces equivalence of convergence cores between semantic fields:

$$L_\Lambda : (\mathcal{S}_i, \mathcal{S}_j) \mapsto (\mathcal{S}'_i, \mathcal{S}'_j) \quad \text{s.t.} \quad \Lambda'_i \approx \Lambda'_j.$$

Definition 5.18 (Operators)

- $\mathcal{P}$: reprojection operator
- $\mathcal{H}$: hope-convergence operator ($\Delta\rho_s > 0$)
- $\mathcal{N}$: non-evaluative protection operator
- $\mathcal{SIL}$: silence/dormancy operator

Definition 5.19 (Λ-sentence) A Λ-sentence is a temporal sequence of these operators:

$$\text{Sentence}_\Lambda := \langle \mathcal{P}, \mathcal{H}, \mathcal{N}, \mathcal{SIL} \rangle.$$

Proposition 5.3 (Necessity of Λ-language) In high-bandwidth civilizations where semantic criticality is inevitable, convergence requires a language that generates shared convergence rather than mere information agreement.

7.10 5.10 Chapter summary

This chapter established:

- Domain Lift and common I/O specifications.
- Connection definitions for AI, education, governance, and economy.
- The operational definition of beyond-global availability.
- The specification and necessity of the Λ-language.

7.11 5.11 Connection to the next chapter

The next chapter summarizes the entire hierarchy as a theorem system and clarifies the ontological position of the Premise-Update OS within civilizational transition.

8 Chapter 6: Unified Propositions and Theorem System — Crystallizing the Premise-Update OS and the Necessary Conditions for Civilizational Transition

8.1 6.1 Purpose: Fix "necessity conditions" while preserving the hierarchy rather than collapsing into a single sentence

In Chapters 1–5, this paper defined the Premise-Update OS through the following components:

- World OS / Personal OS and their connection (update candidate bundle $R(t)$ and connection premises $P(t)$)
- Convergence core Λ, future fixed point F, semantic density tensor P_m
- Pre-judgment structural constraints and the pre-deletion gate P_{crit}
- Separation of reprojection log $\mathbf{R}$ and irreversible log $\mathbf{I}$
- Responsibility coupling via externalization points and minimal signatures $\sigma_{\min}$
- Deletion and disclosure rights $\mathbf{D}$ and the non-evaluative axioms
- Driving states (state machine $\mathsf{S0} \sim \mathsf{S6}$)

- Domain connections (AI / education / governance / economy) and the resulting Λ-language

This chapter re-bundles these elements **by hierarchical level**, fixing a theorem system that clarifies **what becomes inevitable once certain conditions hold**. Here "inevitability" means necessity within the definitional structure rather than a psychological claim.

8.2 6.2 Theorem System I: Necessary conditions of understanding, sharing, and division (independent of bandwidth)

Theorem 6.1 (Bandwidth-independence of understanding) A necessary condition for understanding between agents i and j is not communication bandwidth B but equivalence of convergence cores:

$$\text{Understand}(i, j) \;\Rightarrow\; \Lambda_i \equiv \Lambda_j.$$

Therefore,

$$B \to \infty \text{ and } \Lambda_i \not\equiv \Lambda_j \Rightarrow \text{Understand}(i, j) \text{ is not guaranteed.}$$

Theorem 6.2 (Inevitability of semantic criticality) As bandwidth increases (neural connectivity, AGI outputs, information environment expansion), the pseudo-convergent state $\mathsf{S0}$ becomes unstable and transitions to semantic criticality $\mathsf{S1}$:

$$B \uparrow \Rightarrow \mathsf{S0} \to \mathsf{S1}.$$

Thus high-bandwidth civilization inevitably exposes differences rather than producing perfect understanding.

Corollary 6.1 (Definition of division trajectory) If at semantic criticality $\mathsf{S1}$ convergence-core equivalence does not occur and the semantic density field does not exceed the critical threshold:

$$(\Lambda_i \not\equiv \Lambda_j) \;\wedge\; (\rho_s^{\text{field}} \leq \rho_{\text{crit}}) \Rightarrow \text{division stabilization.}$$

Division is therefore an operational failure of coordinate alignment rather than merely a disagreement of values.

8.3 6.3 Theorem System II: Conditions for the existence of a Premise-Update OS

Definition 6.1 (Existence of a Premise-Update OS) A Premise-Update OS exists when:

1. World OS and Personal OS are connected,

2. the loop

   candidate generation → pre-deletion → selection → externalization → responsibility fixation →

reprojection updating

   operates consistently, and

3. deletion rights **D** and the non-evaluative axioms remain intact.

Theorem 6.3 (Necessary condition: dual-log separation) If reprojection logs **R** and irreversible logs **I** are not separated, the OS degenerates into a surveillance or fixation system.

Therefore:

$$\text{OS existence} \Rightarrow (\mathbf{R} \text{ and } \mathbf{I} \text{ must be separated}).$$

Theorem 6.4 (Necessary condition: externalization-only fixation) If internal updates (premises, values, operations) are fixed irreversibly, Personal OS autonomy is destroyed.

Thus:

$$\text{OS existence} \Rightarrow \text{irreversible fixation restricted to externalization points.}$$

Theorem 6.5 (Necessary condition: minimal signature) If too much information is fixed at externalization points, responsibility verification degenerates into surveillance.

Hence:

$$\text{OS existence} \Rightarrow \sigma_{\min} \text{ must be adopted.}$$

Theorem 6.6 (Necessary condition: non-evaluative axioms) If questions are scored or ranked, premise updating becomes subordinated to competition.

Thus:

$$\text{OS existence} \Rightarrow \forall Q,\ \text{Score}(Q) = \varnothing.$$

Theorem 6.7 (Sufficient condition: closed operational loop) The Premise-Update OS functions operationally when the following simultaneously hold:

1. World OS continuously generates $R(t)$.
2. Personal OS updates $P(t)$ locally.
3. Candidates are pre-filtered by P_m and P_{crit}.
4. Only externalization points are fixed in **I** with $\sigma_{\min}$.
5. Outputs are stored in **R** and used for reprojection updates.

6. Deletion rights **D** remain enforceable.

7. Questions remain non-evaluative.

Under these conditions, the OS becomes an operational machine rather than a conceptual framework.

8.4 6.4 Theorem System III: Isomorphism of domain connections (AI / education / governance / economy)

Theorem 6.8 (Isomorphism of domain connections) Through the mapping Lift_G defined in Chapter 5, any domain G is mapped to the common OS I/O structure:

$$(R_G(t),\ P_m,\ P_{\mathrm{crit}},\ e_G,\ \sigma_{\min},\ \mathbf{R},\ \mathbf{I},\ \mathbf{D}).$$

Therefore:

$$G_\Lambda \cong \mathrm{Proj}_G(\mathrm{OS}_\Lambda).$$

Domain connections are thus projections of a single OS rather than separate ethical frameworks.

Corollary 6.2 (Minimal equivalence of future-causal AI) Future-causal AI is equivalent to AI whose update rules embed the Premise-Update OS.

Corollary 6.3 (Minimal equivalence of non-evaluative education) Non-evaluative education corresponds to connecting question generation to **R** while fixing only externalization events in **I**.

Corollary 6.4 (Minimal equivalence of semantic governance) Semantic governance corresponds to pre-filtering governance candidates, fixing only irreversible institutional outputs, and updating institutional premises via reprojection.

Corollary 6.5 (Minimal equivalence of semantic economy) Semantic economy corresponds to allocating value based on F alignment, semantic density, and accountability traces rather than short-term price indicators.

8.5 6.5 Theorem System IV: Beyond-global usability and civilizational transition

Theorem 6.9 (Three stages of civilization) Civilizational operation can be classified into:

1. C_1: Linguistic civilization (pseudo-convergence dominant)

2. C_2: Neural-connectivity civilization (semantic criticality dominant)

3. C_3: Future-convergence civilization (convergent operation dominant)

Theorem 6.10 (Conditions requiring transition to future-convergence civilization) For neural-connectivity civilization C_2 to remain stable:

$$C_2 \text{ stable} \Rightarrow (\Lambda\text{-sharing}) \wedge (\rho_s^{\text{field}} > \rho_{\text{crit}}).$$

Therefore increasing bandwidth structurally demands the transition toward the Premise-Update OS.

Corollary 6.6 (Necessitation of Λ-language) After semantic criticality, maintaining convergence requires operator-sharing languages (the Λ-language).

8.6 6.6 Final integrated theorem

Theorem 6.11 (Final proposition of the Premise-Update OS) If the World OS continuously generates update candidate bundles $R(t)$ and Personal OSs locally update connection premises $P(t)$, and if increasing bandwidth inevitably produces semantic criticality, then avoiding division and sustaining civilizational operation requires:

- structurally constraining candidate spaces before judgment using P_m and P_{crit},
- fixing only externalization points irreversibly with minimal signatures,
- operating internal updates reversibly through reprojection logs,
- managing state transitions under deletion rights, disclosure control, and non-evaluative axioms.

Under these conditions, the Premise-Update OS (Future-Convergence OS) becomes necessary, and the Λ-language emerges as a consequence of cross-domain connection.

8.7 6.7 Connection to the next chapter

The next chapter completes the definitional structure by providing:

1. a comprehensive definition index,
2. verification that the paper is self-contained,
3. and the minimal set of research questions for measurement, implementation, and validation.

9 Chapter 7: Definition Index, Self-Containment Verification, and Future Research Directions

9.1 7.1 Purpose of the final chapter

The preceding chapters introduced the Premise-Update OS (the Λ-System) as a unified operational structure integrating:

- World OS / Personal OS connection,

- future-convergence cores Λ and future fixed points F,
- semantic density ρ_s and the tensor P_m,
- reprojection logs $\mathbf{R}$ and irreversible logs $\mathbf{I}$,
- responsibility coupling via externalization points and minimal signatures $\sigma_{\min}$,
- deletion and disclosure rights $\mathbf{D}$,
- the driving state machine $\mathsf{S0} \sim \mathsf{S6}$,
- and domain connections across AI, education, governance, and the economy.

The purpose of this final chapter is threefold:

1. To provide a **definition index** summarizing the key constructs fixed in the main text.
2. To verify that the framework is **self-contained** as a definitional system.
3. To outline **future research directions** required for implementation, measurement, and verification.

9.2 7.2 Definition index (core constructs)

OS structure

- Ontological phase of OS
- Five base elements (OS-1 to OS-5)
- Premise-Update OS (Λ-System)

Future-convergence core

- Convergence core Λ
- Future fixed point F
- Semantic field $\mathcal{S}$
- Semantic density ρ_s
- Semantic density tensor P_m

Operational architecture

- World OS and Personal OS
- Update candidate bundle $R(t)$
- Connection premises $P(t)$
- Activation gate ($\Delta S > D(M)$)

Responsibility coupling and log structure

- Reprojection log $\mathbf{R}$
- Irreversible log $\mathbf{I}$
- Externalization point
- Minimal signature $\sigma_{\min}$
- Rights structure $\mathbf{D}$ (deletion and disclosure)

Operational states

- Semantic criticality
- Λ-shift
- Convergent operation
- Silence / exhaustion
- Rights execution
- Externalization fixation

Language and domain connections

- Λ-language operators
- Domain Lift mapping Lift_G
- Future-causal AI (AGI_Λ)
- Non-evaluative education (Edu_Λ)
- Semantic governance (Gov_Λ)
- Semantic economy (Eco_Λ)

These definitions form a minimal conceptual vocabulary enabling consistent reference across the entire framework.

9.3 7.3 Self-containment verification

A definitional framework is self-contained when its operational concepts can be defined without requiring external theoretical assumptions.

This paper satisfies that condition in the following sense:

1. The Premise-Update OS is defined by explicit operational components rather than external metaphysical claims.
2. The log structure $\mathbf{R}$ and $\mathbf{I}$ provides traceability of structural updates.

3. Domain connections are specified via the mapping Lift_G rather than domain-specific reinterpretations.
4. The driving state machine defines minimal transition rules.
5. The theorem system specifies necessary and sufficient conditions within the framework itself.

Therefore the system functions as a **closed definitional architecture**, meaning that every key concept appearing in the text is either:

- directly defined,
- derived from previously defined constructs,
- or introduced as an explicit axiom.

9.4 7.4 Implementation directions

Although the present paper focuses on definitional stabilization, several implementation directions naturally follow.

Computational implementation Future research may explore:

- computational models of semantic density fields,
- algorithmic forms of reprojection updates,
- candidate filtering using tensor-based alignment evaluation,
- and AI architectures embedding reprojection logs.

Institutional implementation Potential institutional experiments include:

- governance systems using minimal signatures for decision traceability,
- education systems recording question logs without evaluation scoring,
- economic accountability structures linking value to responsibility traces.

Measurement and empirical validation Important research questions include:

- operational measurement of semantic density,
- empirical identification of Λ-shift conditions,
- large-scale analysis of reprojection log structures,
- and simulation of semantic criticality in high-bandwidth information environments.

9.5 7.5 Limitations

The present work is limited to definitional and theoretical specification.

Specifically:

- empirical validation is not yet provided,
- formal mathematical proofs of convergence remain open,
- and engineering implementations are only conceptually outlined.

These limitations are intentional: the purpose of the paper is to establish a stable reference framework upon which future theoretical and practical developments may build.

9.6 7.6 Concluding remarks

This paper introduced the Λ-System as a Premise-Update OS and defined the structural conditions under which premise updating becomes operational.

The central insight is that modern high-bandwidth civilizations do not fail primarily because of insufficient information or insufficient computational power, but because systems lack mechanisms for **updating the premises by which decisions are made**.

The Premise-Update OS addresses this by integrating:

- future-convergence constraints,
- semantic density evaluation,
- responsibility coupling through irreversible logs,
- reversible internal updates via reprojection logs,
- and non-evaluative protection of question generation.

Rather than presenting a normative program, the framework aims to provide a stable conceptual architecture for analyzing and implementing premise-updating processes across domains.

Future work will determine whether this architecture can serve as a practical foundation for AI systems, educational environments, institutional governance, and economic coordination in increasingly high-bandwidth societies.